

Capítulo 5 — Red interna y prácticas de desarrollo

Unidad de estudio · Edición ampliada con fundamentos teóricos

Capítulo 5 — Red interna y prácticas de desarrollo	1
5.1. Alcance de la clase	2
5.2. Por qué existe una red interna: origen y diseño.....	2
5.3. El punto de partida: quién ejecuta el código.....	3
5.3.1. Cuándo sí aplica la red interna.....	4
5.4. Anatomía de la red interna	5
5.4.1. El puente, el DNS embebido y el nombre de servicio	5
5.4.2. Qué se gana	5
5.5. Incorporación de la base de datos.....	5
5.5.1. La variable de conexión	6
5.6. El historial	7
5.6.1. Diseño: la persistencia es opcional	7
5.6.2. Los endpoints	7
5.6.3. Por qué el guardado no puede tumbar el cálculo.....	8
5.6.4. Consultas parametrizadas	8
5.7. Despliegue y verificación del aislamiento	8
5.7.1. Verificación funcional.....	9
5.7.2. Verificación del aislamiento.....	9
5.7.3. Estado, volúmenes y persistencia	9
5.8. Prácticas de desarrollo en el repositorio	10
5.8.1. Qué nunca entra a un repositorio.....	10
5.8.2. Ramas	11
5.8.3. Separación de dependencias	11
5.9. Integración continua	11
5.9.1. Protección de la rama principal	12
5.9.2. Despliegue automático.....	13
5.10. Lo que este despliegue todavía no tiene.....	13
5.11. Verificación	14
5.12. Errores frecuentes	14
5.13. Actividades	15
5.14. Síntesis	15
5.15. Referencias y lecturas complementarias	16

5.1. Alcance de la clase

La aplicación está publicada y funcionando. Esta clase incorpora un tercer servicio —una base de datos— y con él **el concepto que da sentido a todo el capítulo de seguridad de la Clase 2**: la comunicación entre servicios sin exponer puertos.

El capítulo tiene dos mitades con una unidad conceptual más fuerte de la que aparenta. La primera introduce la red interna, que es la respuesta arquitectónica al problema que la sección 2.8 dejó planteado: si publicar un puerto lo deja expuesto aunque el firewall diga lo contrario, la solución no es un firewall mejor sino **no publicar el puerto**. La segunda mitad cubre las prácticas de desarrollo que separan un despliegue didáctico de uno sostenible, y que responden a una pregunta emparentada: si el despliegue ya es automático, ¿qué impide que un error llegue a producción tan automáticamente como el resto?

Al finalizar, cada grupo debe tener el historial de operaciones persistido en una base de datos que **no es accesible desde internet**, y un flujo de integración continua que impide desplegar código con pruebas fallidas.

Contenidos

1. Origen y objetivos de diseño del modelo de red de los contenedores.
2. Por qué el frontend no puede usar la red interna.
3. Anatomía de la red interna: el puente, el DNS embebido y el nombre de servicio.
4. Incorporación del servicio de base de datos.
5. El historial: guardar, listar y degradar.
6. Consultas parametrizadas y prevención de inyección.
7. Verificación del aislamiento.
8. Estado, volúmenes y persistencia.
9. Prácticas de desarrollo en el repositorio.
10. Integración continua y despliegue automático.
11. Lo que este despliegue todavía no tiene.

5.2. Por qué existe una red interna: origen y diseño

El capítulo 3 explicó que un contenedor tiene su propio espacio de nombres de red y, por lo tanto, su propia dirección IP. Eso resuelve el aislamiento y crea inmediatamente un problema nuevo: **si cada contenedor tiene su propia dirección y esa dirección cambia, ¿cómo hace uno para encontrar a otro?**

El problema es real y no menor. Las direcciones que Docker asigna salen de un rango privado —habitualmente dentro de `172.17.0.0/16`, uno de los bloques que la RFC 1918 reserva para uso interno— y **se asignan en el orden en que los contenedores arrancan**. Un contenedor que hoy es `172.17.0.3` puede ser `172.17.0.5` mañana, después de un redespigie en otro orden. Escribir esa dirección en un archivo de configuración es garantizar que el sistema se rompa sin que nadie lo toque.

La primera respuesta de Docker, allá por 2014, fue un mecanismo llamado *enlaces* (`--link`): al arrancar un contenedor se declaraba con qué otros quería hablar, y Docker le escribía las direcciones correspondientes en su archivo `/etc/hosts`. Funcionaba, y tenía dos defectos graves: era unidireccional y estático —si el contenedor de destino se reiniciaba con otra dirección, el enlace quedaba apuntando a la nada— y obligaba a declarar el grafo completo de dependencias en la línea de comandos. Está formalmente obsoleto desde hace años y hoy solo se lo encuentra en tutoriales viejos.

El modelo actual, introducido en 2016, se apoya en tres decisiones de diseño que conviene enunciar porque son exactamente lo que este capítulo usa.

Primera decisión: redes definidas por el usuario. En lugar de una única red plana donde todo se ve con todo, se crean redes con nombre, y un contenedor solo puede alcanzar a los que comparten alguna red con él. La red deja de ser un detalle de conectividad y pasa a ser **un límite de alcance**, es decir, una frontera de seguridad. Un proyecto de Easypanel es exactamente una de estas redes.

Segunda decisión: descubrimiento de servicios por DNS. Cada red definida por el usuario trae su propio servidor DNS embebido, que los contenedores consultan en la dirección `127.0.0.11` de su propio espacio de nombres. Ese servidor resuelve **nombres de contenedor y de servicio a la dirección que tengan en ese momento**. El nombre es estable, la dirección no, y nadie tiene que enterarse de los cambios. Es la misma idea de indirección que fundamentaba el capítulo 1 —un nombre estable delante de una dirección inestable— aplicada acá adentro.

Tercera decisión: publicar un puerto es opcional y ortogonal. La comunicación entre contenedores de una misma red **no requiere publicar nada**: el mapeo `-p` sirve exclusivamente para exponer un servicio hacia afuera del anfitrión. Son dos mecanismos distintos que se suelen confundir, y de esa confusión salen la mitad de las bases de datos expuestas a internet del mundo.

PARA ENTENDER

Fijate que el problema es literalmente el mismo del capítulo 1, un piso más abajo: **hay una dirección que cambia y un nombre que no**. Docker resolvió adentro de un servidor el mismo problema que el DNS resolvió para internet entera, con la misma solución, cuarenta años después. Cuando veas ese patrón —un nombre estable delante de algo inestable— ya sabés qué problema está resolviendo.

5.3. El punto de partida: quién ejecuta el código

Antes de usar la red interna conviene responder una pregunta que aparece casi siempre al llegar a este punto: *si el frontend y la API están en el mismo servidor, ¿por qué la API tiene que estar publicada en internet? ¿No podrían hablarse por adentro?*

La respuesta es que **no pueden**, y el motivo es el mismo que se viene señalando desde el `README.md` del proyecto.

El frontend NO se ejecuta en el servidor.
El servidor solo ENTREGA el archivo. El código corre en el navegador del visitante.

	Frontend	Backend
Dónde se ejecuta el código	En el navegador del visitante	En el servidor
Qué red ve	La del visitante, en su casa	La interna del proyecto
¿Puede resolver <code>calculadora_api</code> ?	No	Sí
¿Necesita dominio público?	Sí	Sí

FIGURA 5.1

diagrama

Dos ámbitos enfrentados: la casa del visitante, donde el navegador ejecuta el JavaScript, y el VPS con la red interna del proyecto, donde el contenedor api alcanza al contenedor db por su nombre de servicio. Desde el navegador, el nombre interno calculadora_api no se puede resolver; a la API solo se llega por su dominio público.

Resaltar: la cruz sobre el intento de resolución interna desde el navegador, con la leyenda “el navegador del visitante no está en la red del servidor, y no hay forma de que lo esté”.

Figura 5.1. Ámbito de ejecución y visibilidad de red. El frontend se ejecuta fuera del servidor; por eso la API requiere un dominio público y la base de datos no.

Cuando un visitante en Córdoba abre la calculadora, el `fetch` se ejecuta en **su** máquina. Esa máquina no está en la red interna del VPS y no tiene forma de estarlo: el DNS embebido de la sección 5.2 solo responde a quien esté dentro de la red, y el navegador del visitante está a mil kilómetros. La API debe, por lo tanto, ser alcanzable desde internet.



PARA ENTENDER

Grabate esta pregunta, porque sirve para toda tu carrera: **¿quién ejecuta este código?** Si la respuesta es “el navegador del usuario”, entonces está afuera, no confíes en él, y todo lo que necesite tiene que ser público. Si la respuesta es “un proceso mío en el servidor”, entonces está adentro, y ahí sí podés usar la red interna.

Es la misma pregunta que en el `README.md` explicaba por qué la validación va en los dos lados. La misma pregunta, otra vez, con otra consecuencia.

5.3.1. Cuándo sí aplica la red interna

La red interna es el mecanismo correcto cuando **ambos extremos son procesos del servidor**:

Quién llama	A quién	¿Red interna?
Navegador del visitante	API	No. Requiere dominio público
API	Base de datos	Sí
API	Caché o cola de mensajes	Sí
API	Otro microservicio	Sí

Ese es exactamente el caso que se incorpora ahora.

5.4. Anatomía de la red interna

5.4.1. El puente, el DNS embebido y el nombre de servicio

Cada proyecto de Easypanel constituye una red aislada de Docker, del tipo *puente* (*bridge*) definido por el usuario. Físicamente, esa red es una interfaz virtual en el anfitrión a la que se conectan los contenedores del proyecto, con un rango de direcciones privadas propio. Todos los servicios que la integran pueden comunicarse entre sí por nombre, **sin que ninguno publique puertos hacia el exterior**.

La resolución la hace el servidor DNS embebido de la sección 5.2, y se realiza por **nombre de servicio**, con el formato:

```
<nombre_del_proyecto>_<nombre_del_servicio>
```

Para el proyecto `calculadora` y un servicio de base de datos llamado `db`, el nombre interno es `calculadora_db`. Easypanel expone además dos variables mágicas utilizables en la configuración: `$(PROJECT_NAME)` y `$(SERVICE_NAME)`.

Conviene tener presente que **ese nombre solo existe dentro de esa red**. No es un dominio, no está en ninguna zona DNS pública, y consultarlo desde afuera —o escribirlo en el navegador— no devuelve nada. Que no resuelva no es una limitación: es la propiedad buscada.

5.4.2. Qué se gana

Ventaja	Explicación
El puerto no existe hacia afuera	No hay nada que un firewall deba proteger
No pasa por Traefik	Sin proxy, sin TLS, sin sobrecarga
Menor latencia	El tráfico no sale del servidor
No consume tráfico público	Relevante en planes con cuota de transferencia
Aislamiento entre proyectos	Un servicio de otro proyecto no lo alcanza

La última fila merece subrayarse porque es la que convierte a la red en un mecanismo de seguridad y no solo de conectividad. Si en el mismo VPS conviven el proyecto `calculadora` y otro proyecto cualquiera, sus contenedores **no se ven entre sí**, aunque compartan el mismo servidor, el mismo núcleo y el mismo motor de Docker. Es el principio de mínimo privilegio de la sección 2.12.1, aplicado a la topología de red.



PARA ENTENDER

Volví a la sección 2.8, la de Docker salteándose el firewall. El problema era que publicar un puerto lo dejaba abierto a internet aunque `ufw` dijera lo contrario.

La red interna no resuelve ese problema: lo elimina. Un puerto que nunca se publicó no puede quedar mal protegido, porque no hay nada que proteger. Es la diferencia entre poner un candado bueno y no tener puerta.

5.5. Incorporación de la base de datos

En el proyecto `calculadora`: **+ Service → Postgres**, con el nombre `db`.

Campo	Valor
Nombre del servicio	<code>db</code>
Base de datos	<code>calculadora</code>
Usuario	<code>calculadora</code>
Contraseña	La generada automáticamente
Domains & Proxy	No se configura ninguno
Puerto publicado	Ninguno

FIGURA 5.2

*captura de pantalla de Easypanel**El servicio db con la pestaña Domains & Proxy vacía, sin ningún dominio ni puerto publicado.**Resaltar: un recuadro sobre la sección vacía, con la nota “deliberadamente vacío”.**Difuminar: la contraseña generada.*

Figura 5.2. El servicio de base de datos no declara dominio ni publica puertos.

Las dos últimas filas son el contenido de la clase. Un servicio sin dominio y sin puerto publicado es, desde afuera, **inexistente**: no hay enrutador de Traefik que lo alcance (sección 4.8.1) ni regla de traducción de direcciones que lo exponga (sección 2.8.1). Y sin embargo funciona perfectamente para quien lo necesita.

⚠ OJO ACÁ

No le pongas dominio ni publiques el 5432. En algún momento vas a querer conectarte con DBeaver desde tu notebook y te va a tentar “abrir el puertito nomás”. No lo hagas: un PostgreSQL expuesto a internet lo encuentran los escáneres automáticos en cuestión de horas (sección 2.9.1).

Si necesitás inspeccionar la base, se hace por un túnel SSH:

```
ssh -L 5432:calculadora_db:5432 root@tudominio.com
```

Eso hace pasar la conexión por el canal cifrado de SSH, que ya está autenticado por clave, sin abrir nada nuevo. Es la segunda estrategia de la tabla de la sección 2.8.3, llevada a la práctica.

5.5.1. La variable de conexión

En el servicio `api`, sección Environment, se agrega la cadena de conexión interna:

```
DATABASE_URL=postgres://calculadora:CONTRASEÑA@calculadora_db:5432/calculadora
```

Parte	Valor	Observación
Usuario y contraseña	Los del servicio db	Generados por Easypanel
Host	<code>calculadora_db</code>	Nombre interno, no una dirección IP ni un dominio
Puerto	5432	Interno; no está publicado
Base de datos	<code>calculadora</code>	—

Esta cadena es también un buen ejemplo del principio de configuración de la sección 3.7.2: la misma imagen del backend funciona con base de datos y sin ella, en la notebook y en producción, y lo único que cambia es el valor de una variable de entorno.

📌 DATO

Al crear un servicio de base de datos, Easypanel muestra en su pantalla la **cadena de conexión interna** ya armada, con el nombre de host correcto y las credenciales generadas. Conviene copiarla de ahí en lugar de escribirla a mano: los nombres varían según la versión del panel y ese valor siempre es el vigente.

⚠ OJO ACÁ

Prestá atención al host: **calculadora_db**, no **db.tudominio.com** ni una IP. Ese nombre solo existe dentro de la red del proyecto. Si lo escribís en el navegador no resuelve, y está bien que no resuelva: esa es exactamente la propiedad que queremos.

5.6. El historial

5.6.1. Diseño: la persistencia es opcional

La API se modifica para guardar cada operación y poder listarla. La decisión de diseño más importante es que **la base de datos es opcional**:

DATABASE_URL	Comportamiento de la API
No definida	Funciona exactamente como antes. Sin historial
Definida	Guarda cada operación y permite listarlas

Este patrón tiene nombre en la literatura de sistemas confiables: **degradación elegante**. La idea es que un sistema, ante la falla de un componente, no deje de funcionar sino que reduzca su funcionalidad de forma controlada y previsible. Su contrario es el *fallo en cascada*, en el que la caída de una dependencia secundaria arrastra al servicio entero.

💡 PARA ENTENDER

¿Por qué tanto trabajo para que funcione sin base de datos? Por tres razones concretas: El desarrollo local no necesita levantar un Postgres para tocar una línea de la calculadora. **Las Clases 1 a 4 siguen siendo válidas**. El despliegue que hiciste la semana pasada no se rompe. **Es el patrón correcto**. Una funcionalidad opcional se degrada, no explota. Si la base se cae, la calculadora tiene que seguir calculando. Ese último punto es una decisión de arquitectura de verdad, no un truco didáctico.

5.6.2. Los endpoints

Método	Ruta	Función	Sin base de datos
POST	/api/calcular	Calcula y, si puede, guarda	Calcula igual
GET	/api/historial	Devuelve las últimas operaciones	503 con mensaje explicativo
GET	/api/salud	Estado del servicio y de sus dependencias	persistencia: false

La elección del código **503** no es arbitraria y conviene justificarla, porque la diferencia entre familias de códigos es información de diagnóstico. Un **500** afirma “algo se rompió y no sé qué”: es un error inesperado del servidor. Un **503** afirma “el servicio existe, sé lo que me pedís, y no puedo dárselo ahora porque una dependencia no está disponible”. Lo segundo es verdad y lo primero no. Un cliente que reciba **503** sabe que tiene sentido reintentar más tarde; uno que reciba **500** no sabe nada.

El control de salud pasa a informar el estado de sus dependencias:

```
{ "estado": "ok", "persistencia": true }
```

💡 PARA ENTENDER

Esto último es una práctica estándar que conviene señalar: un *health check* que solo dice “estoy vivo” sirve de poco. El que sirve es el que dice **de qué depende y cómo está cada dependencia**. Cuando tengas monitoreo, esa es la URL que se consulta cada treinta segundos.

5.6.3. Por qué el guardado no puede tumbar el cálculo

La inserción en la base de datos se realiza de forma tal que un fallo no afecte la respuesta al usuario. Si la base no está disponible, la operación se calcula, se responde correctamente y el fallo queda registrado en el log del servidor.

El razonamiento detrás es el análisis de dependencias: **una dependencia es crítica o no lo es, y el código tiene que tratarlas distinto**. Para la calculadora, la base de datos no es crítica: es un registro de lo que ya se hizo. Tratarla como crítica —dejar que su excepción suba y se convierta en un error 500— haría que la disponibilidad del servicio principal quedara atada a la del servicio secundario, que es exactamente el error de diseño que se quiere evitar.

⚠ OJO ACÁ

Este es un criterio de diseño que vale para todo lo que hagas de acá en adelante: **una funcionalidad secundaria nunca puede romper la principal**. Guardar el historial es secundario. Calcular es lo que la aplicación hace. Si por no poder guardar una fila la calculadora deja de calcular, el diseño está mal.

5.6.4. Consultas parametrizadas

La operación de guardado incorpora valores que vienen del exterior, y eso obliga a decir algo sobre cómo se construye una consulta SQL. La regla es breve y no admite excepciones: **los valores nunca se concatenan a la consulta**. Se envían como parámetros, en un canal separado.

```
# Correcto: el valor viaja como parámetro
cur.execute(
    "INSERT INTO historial (operacion, a, b, resultado) VALUES (%s, %s, %s, %s)",
    (operacion, a, b, resultado),
)
```

El detalle que casi siempre se malinterpreta es que **esos %s no son formato de cadena de Python**. No se están reemplazando por texto antes de enviar nada. El controlador envía por un lado la consulta con sus marcadores y por otro los valores, y el motor de base de datos los trata como datos, jamás como instrucciones. Escribir lo mismo con una f-string o con el operador % de Python parece equivalente y es un agujero de seguridad: ahí el valor **sí** se mezcla con la instrucción antes de salir, y quien controle el valor controla la consulta.

Esa clase de falla se llama **inyección** y encabeza desde hace veinte años todas las listas de vulnerabilidades más frecuentes de aplicaciones web. No es un problema exótico: es el más común y el más barato de evitar.

⚠ OJO ACÁ

Cuando escribas SQL, mirá qué separa los datos de la instrucción. Si podés dibujar una línea clara entre las dos cosas —la consulta acá, los valores allá— está bien. Si en algún punto el valor se pegó al texto de la consulta con un +, un % o un f"...", **eso es inyección esperando a alguien que la encuentre**, aunque hoy el valor venga de un campo numérico y te parezca imposible.

5.7. Despliegue y verificación del aislamiento

1. Publicar los cambios en el repositorio del backend.

2. Redespargar el servicio `api`.
3. Verificar en los registros que la conexión a la base se estableció.

5.7.1. Verificación funcional

Comprobación	Resultado esperado
https://api.tudominio.com/api/salud	<code>{"estado":"ok","persistencia":true}</code>
Realizar tres operaciones en la calculadora	Resultados correctos
https://api.tudominio.com/api/historial	Las tres operaciones, más recientes primero

5.7.2. Verificación del aislamiento

Esta es la comprobación central de la clase:

```
nmap -Pn -p 5432 tudominio.com
```

El puerto debe figurar como **cerrado o filtrado**. La base de datos está en pleno funcionamiento, atendiendo consultas de la API, y sin embargo es inalcanzable desde internet.

FIGURA 5.3

composición de dos capturas de pantalla

Lado a lado y del mismo ancho: a la izquierda, el navegador en la ruta del historial mostrando tres operaciones en JSON; a la derecha, una terminal con nmap contra el puerto 5432 informando que está cerrado o filtrado.

Resaltar: un recuadro verde sobre el JSON y uno rojo sobre el estado del puerto.

Figura 5.3. La base de datos atiende consultas de la API y resulta simultáneamente inalcanzable desde internet. Ambas afirmaciones son ciertas al mismo tiempo.

Nótese que esta verificación es del mismo tipo que la de la sección 2.8.2 y por el mismo motivo: es la única que constituye evidencia externa. Cualquier comprobación hecha desde adentro del servidor describiría intenciones.



EXPERIMENTO

Poné las dos ventanas una al lado de la otra: a la izquierda, el historial cargándose perfecto en el navegador; a la derecha, `nmap` diciendo que el 5432 está cerrado.

La base anda y no existe para internet. Esas dos cosas son verdad al mismo tiempo, y ahí es donde se entiende para qué sirve la red interna. Si en toda la clase te llevás una sola imagen, que sea esta.

5.7.3. Estado, volúmenes y persistencia

El servicio de base de datos utiliza un **volumen**, un almacenamiento gestionado por Docker que existe fuera del sistema de archivos del contenedor y sobrevive a su destrucción. Sin él, cada redespigue vaciaría la base.

La razón está en el modelo de la sección 3.3.2: todo lo que un contenedor escribe va a su capa efímera, que se descarta al eliminarlo. Un volumen es, precisamente, un punto del árbol de archivos que **no pertenece a esa capa** sino a un almacenamiento aparte, montado adentro.

De ahí sale una distinción que ordena buena parte de la arquitectura moderna de aplicaciones:

	Servicio sin estado	Servicio con estado
Ejemplos en este proyecto	api, web	db
¿Se puede destruir y recrear?	Sí, sin consecuencias	No sin perder datos
¿Se puede escalar a varias copias?	Sí, trivialmente	Requiere replicación
Dónde vive lo importante	En el repositorio	En el volumen

La regla operativa que se desprende es la que la metodología de los doce factores enuncia como *procesos sin estado*: la aplicación no guarda nada localmente, y todo el estado vive en servicios de respaldo declarados como dependencias. Eso es lo que permite que redespelar la API veinte veces por día no tenga ninguna consecuencia.

⚠ OJO ACÁ

Comprobalo: hacé algunas operaciones, redespelga el servicio `api` y volvé a pedir el historial. Los datos siguen ahí. Y ahora pensá lo que eso implica: a partir del momento en que hay un volumen con datos, **existe algo que se puede perder**. Los contenedores son descartables; el volumen no. Y ahí es donde las copias de seguridad dejan de ser un tema teórico.

5.8. Prácticas de desarrollo en el repositorio

Con la aplicación completa en producción, corresponde revisar cómo se la mantiene.

5.8.1. Qué nunca entra a un repositorio

Nunca se versiona	Dónde va
Contraseñas y cadenas de conexión	Variables de entorno del panel
Claves de API	Variables de entorno del panel
Claves privadas SSH	Solo en el equipo del titular
Archivos <code>.env</code>	En ningún lado; se declaran en el panel
Cachés y entornos virtuales	Se excluyen con <code>.gitignore</code>

Cada repositorio debe tener su propio `.gitignore`. Y conviene notar que `.gitignore` y `.dockerignore` **son archivos distintos con propósitos distintos**: el primero decide qué entra al control de versiones, el segundo qué se envía al contexto de construcción (sección 3.4.1). Un archivo puede estar correctamente excluido de uno y filtrarse por el otro.

⚠ OJO ACÁ

Hasta esta clase, el `.gitignore` del proyecto estaba en el directorio superior, que no es un repositorio. O sea: **no protegía nada**. Que hoy no haya basura versionada es casualidad, no diseño. El primer `git add .` después de correr `pytest` habría subido `__pycache__` y `.pytest_cache`. Cada repositorio necesita el suyo.

⚠ OJO ACÁ

Y algo más serio: **borrar un secreto en un commit posterior no lo elimina**. Queda en el historial y es recuperable por cualquiera que clone. Es el mismo fenómeno que las capas de Docker de la sección 3.3.2: un artefacto inmutable y encadenado no se edita, se tapa.

Si alguna vez subís una credencial, la única respuesta correcta es **rotarla**: darla de baja y generar una nueva. Reescribir el historial es un parche, no una solución.

5.8.2. Ramas

Rama	Función
main	Lo que está en producción. Siempre desplegable
feature/<nombre>	Una funcionalidad en desarrollo

El trabajo se integra a `main` mediante *pull request*, no con un push directo. En un grupo de cuatro personas, esto no es burocracia: es el único mecanismo que evita que dos integrantes pisen el trabajo del otro.

La regla de que `main` esté **siempre desplegable** es más fuerte de lo que parece y es el supuesto sobre el que se apoya toda la sección 5.9. Si la rama principal puede estar rota durante un rato, entonces desplegar automáticamente desde ella es desplegar algo roto; y si no puede estarlo, hace falta un mecanismo que lo garantice, porque la buena voluntad no alcanza.

5.8.3. Separación de dependencias

El archivo `requirements.txt` instala `pytest` y `httpx` en la imagen de producción, aunque solo se necesitan para las pruebas. La corrección consiste en separarlos:

Archivo	Contenido	Se instala en
<code>requirements.txt</code>	<code>fastapi</code> , <code>uvicorn</code> , el conector de base de datos	La imagen de producción
<code>requirements-dev.txt</code>	<code>pytest</code> , <code>httpx</code>	Solo en el entorno de pruebas

No es solo prolijidad: cada paquete instalado en la imagen de producción es peso y es superficie de vulnerabilidades heredada, exactamente como se argumentó al elegir la imagen base en la sección 3.5.1. Una herramienta de pruebas no tiene ninguna razón para estar corriendo en un servidor expuesto a internet.

5.9. Integración continua

Hasta acá, cualquier push a `main` llega a producción sin que nadie ejecute las pruebas. Un cambio defectuoso se despliega igual.

La práctica que corrige eso se llama **integración continua** y es más vieja que las herramientas que la implementan: se formuló en los años noventa, en el contexto de la programación extrema, con una premisa contraintuitiva para la época. Si integrar el trabajo de varias personas es doloroso, la respuesta no es integrar menos seguido sino **integrar mucho más seguido**, en fragmentos chicos, con verificación automática en cada integración. El dolor de integrar crece con el tamaño del cambio; achicando el cambio, desaparece.

Una **acción de GitHub** ejecuta la suite de pruebas en cada push y en cada pull request, y marca el resultado.

```

name: tests

on:
  push:
    branches: [main]
  pull_request:

jobs:
  pytest:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - run: pip install -r requirements.txt -r requirements-dev.txt
      - run: pytest -q

```

FIGURA 5.4

captura de GitHub

Un pull request con la verificación de pruebas resuelta. Lo ideal es presentar dos estados juntos: uno en rojo con la prueba fallando y el botón de integración bloqueado, y otro en verde tras la corrección.

Resaltar: el mensaje de bloqueo de la integración en la captura en rojo.

Figura 5.4. La verificación automática impide integrar código con pruebas fallidas.

Obsérvese que el flujo instala las dependencias **en una máquina limpia**, distinta de la de cualquier integrante. Esa es una propiedad valiosa por sí sola, independiente de las pruebas: detecta las dependencias que alguien tiene instaladas localmente y se olvidó de declarar. Un proyecto que solo se construye en la máquina de quien lo escribió no está terminado, y es el mismo problema que motivó los contenedores en la sección 3.2.

5.9.1. Protección de la rama principal

La acción por sí sola informa, pero no impide nada. Para que efectivamente bloquee, se configura en **Settings → Branches** una regla de protección sobre main que exija la verificación en verde antes de permitir la integración.

La distinción entre *informar* y *bloquear* es la que hace que el mecanismo funcione. Una advertencia que se puede ignorar se ignora, sobre todo bajo presión de entrega, que es precisamente cuando más falta hace. Un control que bloquea no depende de la disciplina de nadie.



PARA ENTENDER

Acá se cierra un círculo que arrancó en la primera clase de la materia. Cuando escribiste el primer test te preguntaste para qué servía, si el código andaba.

Sirve para esto: **para que una máquina revise por vos, cada vez, sin cansarse y sin confiar en tu memoria.** Un test que corrés solo cuando te acordás no es una red de seguridad. Un test que corre automáticamente y bloquea la integración, sí.

5.9.2. Despliegue automático

Easypanel permite configurar un *webhook* para que GitHub le avise de cada push y el servicio se redesplice solo. Con eso, la cadena queda completa: alguien integra un cambio, se ejecutan las pruebas, se aprueba, se fusiona, y el cambio está en producción sin que nadie toque un botón.

Conviene nombrar la distinción que la industria hace acá, porque los dos términos se usan como sinónimos y no lo son. **Entrega continua** significa que todo cambio integrado queda *listo para desplegar*, y el despliegue lo dispara una persona. **Despliegue continuo** significa que se despliega solo. Lo que se configura en esta sección es lo segundo, que es apropiado para un práctico y para muchos productos reales, y que exige más confianza en la verificación automática.

⚠ OJO ACÁ

El orden correcto es: primero la integración continua, después el despliegue automático. Si conectás el despliegue automático sin tener las pruebas corriendo, lo único que lograste es automatizar la llegada de errores a producción, más rápido y con menos supervisión. La automatización sin verificación no es una mejora.

5.10. Lo que este despliegue todavía no tiene

Corresponde ser explícito sobre las limitaciones del resultado obtenido, ordenadas por prioridad.

Falta	Consecuencia	Cuándo pasa a ser urgente
Copias de seguridad	Una pérdida de datos es definitiva	Ya. Desde que existe la base
Supervisión	Una caída se descubre cuando alguien avisa	Cuando alguien dependa del servicio
Limitación de tasa	Cualquiera puede saturar la API	Cuando cada petición tenga un costo
Entorno de pruebas	Se prueba en producción	Cuando haya usuarios reales
Registro centralizado	Los registros se pierden al redesplicar	Al investigar un incidente pasado
Gestión de secretos	Las credenciales viven en el panel	Con más de un entorno

Sobre la primera fila vale una precisión operativa, porque es la única que ya está vencida. La regla clásica de resguardo se enuncia como **3-2-1**: tres copias de los datos, en dos medios distintos, con una fuera del sitio. En este despliegue hay exactamente **una** copia, en un medio, en el mismo lugar. Y la parte que más se descuida no es hacer la copia sino **probar la restauración**: una copia que nunca se restauró no es una copia de seguridad, es una suposición.

La quinta fila conecta con un principio de la metodología de los doce factores que conviene mencionar: los registros se tratan como un **flujo de eventos** que el proceso escribe a la salida estándar, y es la plataforma la que decide dónde almacenarlos. Es exactamente lo que hacen los contenedores del proyecto, y es la razón por la que `docker logs` funciona sin que nadie haya configurado nada. Lo que falta no es producir los registros: es no perderlos.

**PARA ENTENDER**

Ninguna de estas cosas hacía falta para aprender el flujo. **Todas hacen falta el día que alguien dependa de que esto funcione.** Y esa transición no se anuncia: un día alguien empieza a usar lo que hiciste y ya estás en producción de verdad, con o sin copias de seguridad.

5.11. Verificación

#	Comprobación	Resultado esperado
1	/api/salud	persistencia: true
2	Tres operaciones y consulta del historial	Las tres, más recientes primero
3	nmap -Pn -p 5432 tudominio.com	Cerrado o filtrado
4	Redespliegue de api y nueva consulta del historial	Los datos persisten
5	Quitar DATABASE_URL y redesplegar	La calculadora sigue calculando
6	/api/historial sin base de datos	503 con mensaje explicativo
7	Pull request con una prueba rota	La verificación falla y bloquea
8	git status tras correr pytest	Sin archivos de caché sin seguimiento

Las comprobaciones 3 y 5 son las conceptualmente importantes: la primera demuestra el aislamiento de la sección 5.4, la segunda demuestra la degradación de la sección 5.6.1. Son las dos ideas del capítulo, cada una reducida a un comando.

5.12. Errores frecuentes

Síntoma	Causa	Resolución
could not translate host name	Nombre de host interno mal escrito	Copiar la cadena que muestra el panel
connection refused al iniciar la API	La base todavía no terminó de arrancar	Reintento con espera; verificar el orden de arranque
persistencia: false con la variable cargada	Falta redesplegar	Botón Deploy
El historial se vacía en cada despliegue	El servicio no tiene volumen	Revisar el almacenamiento del servicio db
nmap muestra el 5432 abierto	Se publicó el puerto	Quitar la publicación en la configuración del servicio
El nombre interno no resuelve desde otro proyecto	Son redes distintas: es el comportamiento esperado	Mover el servicio al mismo proyecto
La acción de GitHub falla al instalar	Falta requirements-dev.txt	Crear el archivo y referenciarlo
Las pruebas pasan en local y fallan en la acción	Dependencia instalada solo localmente	Declararla en el archivo correspondiente

5.13. Actividades

Actividad 1 — Demostración del aislamiento. Documentar con capturas que el historial funciona y que el puerto 5432 está cerrado desde el exterior. Redactar en tres oraciones por qué ambas cosas son ciertas simultáneamente, apoyándose en el modelo de la sección 5.4.1.

Actividad 2 — Degradación. Quitar `DATABASE_URL`, redespargar y verificar que la calculadora sigue funcionando. Documentar la respuesta de `/api/salud` y la de `/api/historial`, y justificar por qué el código devuelto es 503 y no 500 ni 404.

Actividad 3 — Persistencia. Cargar operaciones, redespargar el servicio `api`, y luego el servicio `db`. Determinar en cuál de los dos casos sobreviven los datos y explicar por qué, en términos de la capa efímera y el volumen de la sección 5.7.3.

Actividad 4 — Integración continua. Abrir un pull request que rompa deliberadamente una prueba, verificar que la verificación falla, corregirlo y verificar que pasa. Comprobar además qué ocurre si la regla de protección de rama está desactivada: ¿se puede fusionar igual?

Actividad 5 — Auditoría del repositorio. Revisar ambos repositorios y confeccionar una lista de todo lo que no debería estar versionado. Corregir los `.gitignore` en consecuencia. Verificar por separado el `.dockerignore` y explicar por qué son dos archivos distintos.

Actividad 6 — El descubrimiento de servicios, observado. (*requiere acceso al servidor*) Desde una consola dentro del contenedor `api` (`docker exec -it <id> sh`), resolver el nombre `calculadora_db` y anotar la dirección obtenida. Reiniciar el servicio `db`, volver a resolver y comparar. Explicar qué cambió, qué no, y por qué la cadena de conexión no necesitó modificarse.

Actividad 7 — Integradora. Agregar la operación potencia siguiendo el ciclo completo: rama, prueba primero, implementación, pull request, verificación en verde, integración y despliegue. Documentar cada paso e indicar en cuáles intervino una verificación automática.

5.14. Síntesis

1. Los contenedores tienen direcciones **inestables** y nombres **estables**. El DNS embebido de cada red resuelve los segundos a las primeras: es el mismo problema del capítulo 1, un piso más abajo.
2. La pregunta que ordena todo es **quién ejecuta el código**. El frontend se ejecuta en el navegador del visitante; por eso la API necesita dominio público.
3. La red interna aplica cuando **ambos extremos son procesos del servidor**, y funciona como **límite de alcance**: dos proyectos en el mismo servidor no se ven.
4. **El puerto más seguro es el que nunca se publicó**. La red interna no protege el puerto: hace que no exista.
5. Una funcionalidad secundaria **se degrada, no explota**. Y el código de estado debe decir la verdad: 503 cuando falta una dependencia, no 500.
6. Un *health check* útil informa **el estado de las dependencias**, no solo el propio.
7. En SQL, **los valores nunca se concatenan**: viajan como parámetros, separados de la instrucción. Es la falla más común y la más barata de evitar.
8. Los servicios **sin estado** son descartables; el estado vive en volúmenes, y desde que hay un volumen con datos hay algo que se puede perder.
9. Los secretos **nunca se versionan**, y borrarlos en un commit posterior no los elimina: hay que rotarlos.
10. **Integración continua antes que despliegue automático**. Automatizar sin verificar solo acelera los errores. Y un control que informa se ignora: el que sirve es el que bloquea.

5.15. Referencias y lecturas complementarias

Sobre el modelo de red de los contenedores, la referencia operativa es la documentación oficial de Docker en docs.docker.com/network, que describe los tipos de red, el servidor DNS embebido y el descubrimiento por nombre. Los rangos de direcciones que se utilizan están reservados por la **RFC 1918** (*Address Allocation for Private Internets*). La metodología de los **doce factores** (12factor.net) aporta tres de los principios que este capítulo aplica: el IV (*Backing services*), que trata a la base de datos como un recurso adjunto configurable; el VI (*Processes*), del que sale la distinción entre servicios con y sin estado; y el XI (*Logs*), que fundamenta el tratamiento de los registros como flujo de eventos.

En seguridad de aplicaciones, el catálogo de referencia es el **OWASP Top Ten**, cuya categoría de *inyección* fundamenta la sección 5.6.4; la *SQL Injection Prevention Cheat Sheet* del mismo proyecto es la guía práctica más directa sobre consultas parametrizadas. La documentación de `psycpg`, en psycpg.org/psycpg3/docs, explica con precisión por qué sus marcadores de posición no son formato de cadena.

Sobre entrega de software, el texto canónico es J. Humble y D. Farley, *Continuous Delivery* (Addison-Wesley, 2010), que define el flujo de integración y despliegue que esta clase implementa en pequeño. Para los patrones de estabilidad —degradación elegante, fallos en cascada, tiempos de espera y reintentos—, M. Nygard, *Release It!* (2.ª edición, Pragmatic Bookshelf, 2018) es la referencia, y la sección 5.6.3 es una aplicación directa de su primer capítulo. Para el diseño de sistemas con estado, replicación y garantías de persistencia, M. Kleppmann, *Designing Data-Intensive Applications* (O'Reilly, 2017) es hoy el manual estándar. Finalmente, *Site Reliability Engineering* (Beyer et al., O'Reilly, 2016, disponible gratuitamente en sre.google/books) desarrolla en profundidad los temas de la sección 5.10: supervisión, respuesta a incidentes y qué significa realmente operar un servicio del que alguien depende.